

AIML Workflow Management for Datamak

LLM and agent options for supervising the origin-centric workflow

March 27, 2026

Current starting point

Datamak now has an explicit origin-centric workflow in the GUI, with structured workflow state, action dispatching, and remote status tracking. This makes an AI workflow supervisor feasible.

Why this is now realistic

- ▶ The workflow is no longer hidden in scripts only. The main stages are visible in the Workflow panel.
- ▶ Actions are explicit and tool-like in `gui/actions.py`.
- ▶ Per-origin state is already summarized from `data_equil`, `gk_study`, `gk_input`, `gk_batch`, `gk_run`, and `flux_action_log`.
- ▶ Datamak already contains a first rule-based assistant: `get_ai_suggestions()`.

Main design consequence

The AI layer should supervise a *known workflow* rather than guess how the workflow works.

Recommended contract

Datamak remains the source of truth and executor. The model reads a compact workflow-state summary and selects among a small, explicit set of allowed actions.

- ▶ **Good:** model explains the current state, recommends the next step, or calls a whitelisted action tool.
- ▶ **Avoid:** free-form shell generation, free-form SQL mutation, or letting the model infer workflow semantics from logs alone.
- ▶ Remote or mutating actions should remain gated by user confirmation in early versions.

Option 1: AI advisor

Behavior

Read-only assistant attached to the selected `data_origin`. It does not run actions; it diagnoses the workflow and recommends the next step.

Input

- ▶ selected origin id
- ▶ compact workflow-state JSON
- ▶ short policy prompt

Output

- ▶ summary
- ▶ risks or blockers
- ▶ recommended next action
- ▶ short explanation
- ▶ confidence

Why start here

Lowest risk, fastest to implement, and immediately useful to operators.

Option 2: Tool-calling workflow supervisor

Behavior

The model can call Datamak tools, but only through a narrow, typed interface.

Typical tools

Tool	Role
<code>get_origin_workflow_state</code>	Return compact origin status JSON
<code>list_allowed_actions</code>	Return only actions valid for the current origin
<code>check_flux_status</code>	Refresh Flux job state
<code>run_on_flux</code>	Submit step-2 workflow
<code>sync_back_from_flux</code>	Import staged Flux results
<code>check_simulations</code>	Refresh batch and run monitoring

- ▶ Read-only tools may be autonomous.
- ▶ Mutating or remote tools should initially require approval.

Option 3: Async agent

- ▶ Useful when the reasoning itself is longer or multi-step:
 - ▶ “Analyze all origins and propose an execution plan”
 - ▶ “Monitor this origin until it is safe to sync”
 - ▶ “Read failure logs and prepare a recovery plan”
- ▶ Better fit for a backend service than for a blocking Flask request.
- ▶ Can be integrated using OpenAI Responses API background mode and webhooks or polling.

Implication

This is the right architecture for a true supervisor, but not for v1.

Can a small local model on a laptop do this?

Yes, for narrow tasks

- ▶ summarize workflow state
- ▶ rank the next action among a small list
- ▶ classify failures or stalled states
- ▶ explain logs in plain language

No, not reliably for

- ▶ unattended orchestration
- ▶ ambiguous recovery decisions
- ▶ shell or SQL generation
- ▶ broad tool calling without strong guardrails

Practical split

Use a local small model for *advisor/explainer* mode, and a stronger cloud reasoning model for *tool-calling supervisor* mode.

Codex is best used as a development agent

Codex is a strong fit for evolving the Datamak repository, not as the primary runtime workflow controller inside the product.

- ▶ Use Codex for
 - ▶ adding workflow tools
 - ▶ refactoring the GUI and controller
 - ▶ writing prompts, docs, and tests
 - ▶ auditing logs and proposing implementation changes
- ▶ Do not use Codex as the live operator loop
 - ▶ The runtime product should call Datamak tools directly.
 - ▶ The workflow agent should consume Datamak state, not repo internals.

Suggested v1 architecture

A good first slice

Add an **AI Workflow Advisor** section to the Workflow panel.

1. User selects a `data_origin`.
2. Backend builds a compact JSON summary of that origin.
3. The model returns a short response:
 - ▶ workflow summary
 - ▶ blockers
 - ▶ recommended next action
 - ▶ short reasoning
 - ▶ confidence
4. User decides whether to click the real Datamak action button.

Why this is the right v1

Immediate value, preserved operator control, and low workflow corruption risk.

Suggested v2 architecture

Add typed Datamak tools

Promote the advisor to a supervisor with a short tool list and explicit approval gates for mutating actions.

- ▶ keep action names aligned with `gui/actions.py`
- ▶ expose only valid actions for the selected origin
- ▶ log each AI recommendation and tool invocation
- ▶ keep a human-visible execution trace in the GUI

Non-negotiable guardrails

No raw shell access, no free-form SQL, no broad hidden autonomy, and no action execution without origin-aware validation.

Recommended roadmap

Stage	Outcome
1	AI advisor inside the Workflow panel
2	Compact origin-state API and eval set for recommendation quality
3	Typed Datamak tools for read-only and approved actions
4	Tool-calling supervisor with approval gates
5	Async backend worker using background responses and webhook or polling

Bottom line

The workflow is now explicit enough that AIML workflow management is worth building. Start with *advisor* mode, then move to *supervisor* mode once the tool contract and evals are stable.